

복사전달 특강

Special Topics in Radiative Transfer

Lecture 14
2026 June 2 (Tuesday), 2PM

updated on 05/25

Based on: Numba documentation, Seon 2009 (PKAS), Amanatides & Woo 1987

선광일 (Kwangil Seon)
KASI / UST

Outline – Numba JIT and Fast Ray Traversal

Numba JIT: Where to Place the Boundary

Ray Traversal on a Cartesian Density Grid

Summary

Why Numba?

Monte Carlo radiative transfer hot loops are deeply nested, scalar-heavy, and not vectorizable in NumPy — exactly the workload that pure-Python loops execute slowest on.

Numba's job. Compile a Python function to machine code via LLVM, ahead of the first call. A well-placed `@njit` delivers C-level performance on the same source code, while letting the orchestrating code stay in plain Python.

The catch. Inside a `@njit` function, every callee must itself be Numba-compilable: another `@njit` function, or a NumPy / built-in operation that Numba understands natively. A forgotten decorator on a helper is the single most common cause of failure.

The JIT boundary rule

Three lines summarize where to put `@njit`:

1. The function that contains the *hot loop* (where most of the CPU time is spent) must carry `@njit`.
2. Every helper called from inside that hot loop must also carry `@njit`.
3. The outer Python wrapper that orchestrates calls into the JIT'd code does *not* need `@njit`.

Boundary cost. Crossing the Python / JIT boundary is a few μ s per call. If each JIT call does more than a few ms of work, the boundary cost is invisible. A tight Python loop that calls a fast JIT function 10^6 times *is* dominated by the boundary; the fix is to lift the loop into the JIT region (§ JIT outer wrapper).

Imports and decorator options

The minimum import is just the decorator:

```
from numba import njit
```

For parallel loops, also import prange:

```
from numba import njit, prange
```

The decorator options used in this repository:

- ▶ `@njit(cache=True)` — write the compiled binary to `__pycache__/*.nbc`; next session skips compilation. Recommended when the signature is stable.
- ▶ `@njit(parallel=True)` — enable Numba's parallel transforms; this is what makes `prange` multithread.
- ▶ `@njit(fastmath=True)` — relax IEEE-754 (allow reassociation, no signed zero, ...). Use sparingly: it can change numerical results.

A minimal example: π by dart-throwing

```
import numpy as np
from numba import njit

@njit(cache=True)
def sample_point()::                                # helper: must be @njit
    x = 2.0 * np.random.random() - 1.0
    y = 2.0 * np.random.random() - 1.0
    return x, y

@njit(cache=True)
def count_inside(N, seed=42):                        # hot loop: must be @njit
    np.random.seed(seed)
    count = 0
    for _ in range(N):
        x, y = sample_point()
        if x*x + y*y < 1.0:
            count += 1
    return count

def estimate_pi(N, seed=42):                         # plain-Python wrapper
    return 4.0 * count_inside(N, seed) / N
```

`sample_point` is inlined into the hot loop; `estimate_pi` crosses the JIT boundary once.

Compilation cost and warm-up

The *first* call to a `@njit` function in a Python process triggers LLVM compilation: ~ 0.1 s for a trivial helper, several seconds for `parallel=True` code. Subsequent calls execute the compiled binary directly. With `cache=True`, the next session loads the cached binary in a few tens of ms.

Warm-up is cosmetic, not load-bearing

```
print("Warming up JIT ...")
_ = my_jitted_function(small_problem_size, seed=0)
```

Compilation happens once per session in any case. The warm-up call only shifts *when* it happens — out of the timed region and into a labelled prelude. Output is bit-for-bit identical with or without warm-up; total wall time is unchanged.

When to skip warm-up. Production scripts and CI jobs that don't time anything do not benefit. The real persistent speedup across sessions is `cache=True`, not warm-up; with caching, even the warm-up call itself is essentially free.

When to JIT the outer wrapper

Two situations only:

1. **The outer loop is itself the hot loop.** If a Python for calls a very fast JIT function many times, the per-call boundary cost ($\sim 1 \mu\text{s}$) dominates. Putting `@njit` on the wrapper folds the loop into the JIT region and removes the crossings.
2. **You want Numba parallelism.** A `prange` loop only parallelises inside `@njit(parallel=True)`. The outer wrapper is the natural place to declare it.

The parallel pattern, extending the π example to many independent runs at once:

```
from numba import njit, prange

@njit(cache=True, parallel=True)
def count_inside_many(N_per, N_runs, base_seed=42):
    counts = np.empty(N_runs, dtype=np.int64)
    for i in prange(N_runs):                # threaded
        counts[i] = count_inside(N_per, base_seed+i) # inner @njit
    return counts
```

The outer JIT is needed because of `prange`; the inner `count_inside` remains `@njit` because it is called from JIT code.

Common pitfalls

1. **Every callee inside @njit must also be @njit.** The single most common compile-time error. Numba's error message points at the offending callee.
2. **No Python objects in hot paths.** Lists, dicts, and arbitrary classes are unsupported (or use Numba's slower typed containers). Stick to NumPy arrays and scalars.
3. **Avoid allocations in the inner loop.** Returning a tuple of scalars is allocation-free; returning a small NumPy array per call allocates on every iteration and shows up immediately in profiles. The idiom is to pass and return scalar components.
4. `np.random` **inside @njit uses Numba's own RNG state**, independent of NumPy's global generator. To reproduce a run, call `np.random.seed(seed)` from *inside* the JIT function.
5. `cache=True` **can serve a stale binary.** If a function still runs the old version after editing, delete `__pycache__/*.nbc` and rerun.

Why ray traversal matters in MCRT

For every free flight and every peeling-off direction, an MCRT code integrates the optical depth

$$\tau(\ell) = \int_0^\ell \kappa(x, y, z) d\ell$$

along a straight photon path through a piecewise-constant medium. In an optically thick cloud this kernel runs many times per packet and is the *single largest contributor* to the per-photon cost.

Two algorithms, same answer.

- ▶ **Baseline** (Seon 2009): recompute $\delta X, \delta Y, \delta Z$ from the current position each iteration. Three subtractions and three divisions per step.
- ▶ **Fast Voxel Traversal** (Amanatides & Woo 1987): incremental Digital Differential Analyzer (DDA). Two comparisons + one addition + one integer step per iteration.

Both visit the same voxels in the same order. The DDA simply spends much less arithmetic per step.

Grid setup

Volume $[X_{\min}, X_{\max}] \times [Y_{\min}, Y_{\max}] \times [Z_{\min}, Z_{\max}]$ partitioned into $N_x \times N_y \times N_z$ uniform cells:

$$\Delta X = \frac{X_{\max} - X_{\min}}{N_x}, \quad \Delta Y = \frac{Y_{\max} - Y_{\min}}{N_y}, \quad \Delta Z = \frac{Z_{\max} - Z_{\min}}{N_z}.$$

Cell (i, j, k) corner: $X_i = (i - 1)\Delta X + X_{\min}$, opposite corner $(X_{i+1}, Y_{j+1}, Z_{k+1})$. Opacity per unit length

$$\kappa(i, j, k) \equiv \sigma n(\bar{x}, \bar{y}, \bar{z})$$

is constant on each cell. Photon at $(X_{\text{cur}}, Y_{\text{cur}}, Z_{\text{cur}})$ in cell (i, j, k) , moving along $\mathbf{n} = (n_x, n_y, n_z)$, $|\mathbf{n}| = 1$. Find step length L with

$$\int_0^L \kappa(x(\ell), y(\ell), z(\ell)) d\ell = \tau_{\text{target}} = -\ln \xi.$$

Because κ is piecewise constant, $\tau(\ell) = \sum_p \kappa(i_p, j_p, k_p) \delta L_p$.

The cell-crossing geometry

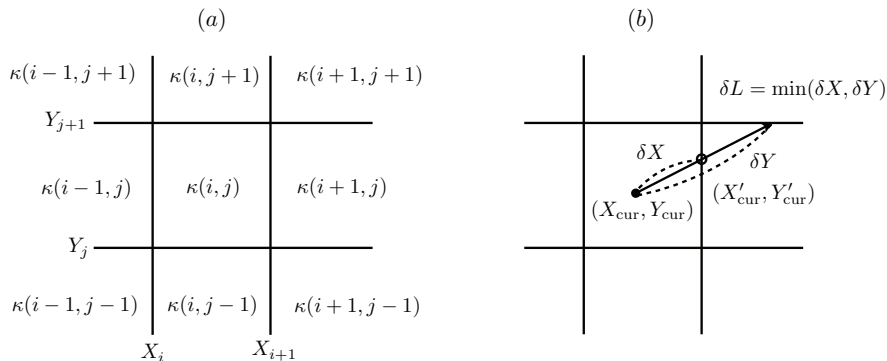


Figure 1: Reproduced from Seon (2009), Fig. 3. **(a)** 2D sketch of the Cartesian grid; cell (i, j) stores $\kappa(i, j)$ and is bounded by the planes $x = X_i, X_{i+1}$ and $y = Y_j, Y_{j+1}$. **(b)** One traversal step: from $(X_{\text{cur}}, Y_{\text{cur}})$ along direction (n_x, n_y) , the parametric distances to the next x - and y -faces are δX and δY ; the actual step is $\delta L = \min(\delta X, \delta Y)$, so the ray exits through whichever face it reaches first.

Baseline algorithm (Seon 2009)

Distances to the next axis-aligned faces from $(X_{\text{cur}}, Y_{\text{cur}}, Z_{\text{cur}})$:

$$\delta X = \begin{cases} (X_{i+1} - X_{\text{cur}})/n_x, & n_x > 0, \\ (X_i - X_{\text{cur}})/n_x, & n_x < 0, \end{cases} \quad \delta Y, \delta Z \text{ analogously,}$$

$$\delta L = \min(\delta X, \delta Y, \delta Z).$$

Advance the photon:

$$X_{\text{cur}} += n_x \delta L, \quad Y_{\text{cur}} += n_y \delta L, \quad Z_{\text{cur}} += n_z \delta L.$$

Increment the cell index along the winning axis ($i += \pm 1$, etc.). Accumulate $\tau_{\text{acc}} += \kappa(i, j, k) \delta L$ and repeat.

When $\tau_{\text{acc}} + \kappa \delta L \geq \tau_{\text{target}}$, the interaction lies inside the current cell; solve the partial step

$$\delta L^* = \frac{\tau_{\text{target}} - \tau_{\text{acc}}}{\kappa(i, j, k)}, \quad x' = X_{\text{cur}} + n_x \delta L^*, \dots$$

Cost per step: 3 div + 3 sub + 4 mul + 4 add + 2 cmp, dominated by the three divisions.

The DDA key observation

Along a fixed direction $\mathbf{n}$, the parametric distance between consecutive x -faces is a *constant*:

$$t_{\delta_x} \equiv \frac{\Delta X}{|n_x|} \quad (\text{and similarly } t_{\delta_y}, t_{\delta_z}).$$

So once the parametric distance to the *first* x -face is known, every subsequent x -face distance follows by repeated addition of t_{δ_x} . No subtraction or division is ever required inside the loop.

Ray parameterization. Write $\mathbf{p}(t) = (X_{\text{cur}}, Y_{\text{cur}}, Z_{\text{cur}}) + t\mathbf{n}$, $t \geq 0$. Because $|\mathbf{n}| = 1$, t is the physical arc length along the ray; δL_p is just the difference of two consecutive t values.

DDA: one-time initialization

At the start of a ray, compute and store:

$$\text{step}_x = \text{sign}(n_x), \quad \text{step}_y = \text{sign}(n_y), \quad \text{step}_z = \text{sign}(n_z),$$

$$t_{\max}^x = \begin{cases} (X_{i+1} - X_{\text{cur}})/n_x, & n_x > 0 \\ (X_i - X_{\text{cur}})/n_x, & n_x < 0 \end{cases} \quad (\text{and } t_{\max}^y, t_{\max}^z),$$

$$t_{\delta x} = \Delta X/|n_x|, \quad t_{\delta y} = \Delta Y/|n_y|, \quad t_{\delta z} = \Delta Z/|n_z|.$$

Cost: 6 div + 3 sub *once per ray*. This is the same arithmetic the baseline does on *every* step — the DDA amortizes it across the whole ray.

Axis components with $n_x = 0$ set the corresponding $t_{\max}$ and t_{δ} to $+\infty$ so they are never selected as the minimum.

DDA: inner loop

Maintain t_{cur} (initially 0) and τ_{acc} (initially 0). The 3D loop body is a three-way minimum implemented as cascaded comparisons:

```
if tMaxX < tMaxY:
    if tMaxX < tMaxZ:          axis, tHit = 'X', tMaxX
    else:                     axis, tHit = 'Z', tMaxZ
else:
    if tMaxY < tMaxZ:          axis, tHit = 'Y', tMaxY
    else:                     axis, tHit = 'Z', tMaxZ

dL      = tHit - t_cur
dtau     = kappa[i, j, k] * dL
if tau_acc + dtau >= tau_target:      # interaction inside cell
    dL_star = (tau_target - tau_acc) / kappa[i, j, k]
    return reconstruct_position(t_cur + dL_star)

t_cur    = tHit                      # advance running t
tau_acc += dtau

if axis == 'X': i += stepX; tMaxX += tDeltaX      # one add, no div
elif axis == 'Y': j += stepY; tMaxY += tDeltaY
else:          k += stepZ; tMaxZ += tDeltaZ
```

Per-step cost: 2 cmps, 1 sub (δL), 1 mul + 1 add (τ), 1 add (t_{max} refresh), 1 integer step. *No divisions inside the loop.*

How much faster? – operation counts

Operation per voxel-traversal step	Baseline	DDA
Divisions	3	0
Subtractions	3	1
Additions	3	2
Multiplies	4	1
FP comparisons	2	2
Integer increments	1	1
Approx. FLOP / step	~ 13 (incl. 3 div)	~ 6 (no div)

Cycle model. With pipelined ops at ~ 1 cycle and division latency $c_{\text{div}} \approx 25$ cycles (double precision `vdivpd`),

$$C_{\text{base}} \approx 3 \times 25 + 10 \times 1 = 85 \text{ cycles}, \quad C_{\text{DDA}} \approx 0 + 6 \times 1 = 6 \text{ cycles}.$$

Per-step speedup $\boxed{C_{\text{base}}/C_{\text{DDA}} \sim 10\text{--}15}$. Where division is fully pipelined the gap narrows to $\sim 3\text{--}5$; it never depends on ray length or optical depth.

Break-even and asymptotic cost

Initialization cost. The DDA pays one-time $6 \text{ div} + 3 \text{ sub} \approx 150$ cycles, vs. essentially zero for the baseline.

Per-step saving. $C_{\text{base}} - C_{\text{DDA}} \approx 80$ cycles.

Break-even.

$$M_{\text{BE}} \approx \frac{150}{80} \approx 2 \text{ steps.}$$

Any ray that crosses three or more voxels favors the DDA. In MCRT the typical photon crosses many cells before its first scattering, so initialization is irrelevant in practice.

Asymptotic. Both algorithms visit the same $M = O(\tau/(\kappa \Delta X))$ voxels per ray, so total cost is $MC + C_{\text{init}}$. The DDA shrinks C by an order of magnitude and leaves M unchanged. In a code whose hot loop is the voxel traversal — most pure-scattering codes and the optically thick limit of absorption-dominated codes — the overall MCRT wall-clock improves by a similar factor.

- ▶ **Snap-to-face is automatic in the DDA.** The cell index changes by an integer ± 1 ; the physical position is only reconstructed at the end from $t_{\text{cur}} + \delta L^*$. There is no opportunity for the index and position to disagree — a free robustness gain over the baseline's manual snapping.
- ▶ **Forced first scattering.** (Witt 1977) Run either algorithm with an unbounded target until the ray exits the volume and read off the accumulated τ_1 . The DDA is the natural choice because every ray-to-boundary crosses many voxels.
- ▶ **Peeling-off optical depth.** (Yusef-Zadeh et al. 1984) Once per scattering, per observer direction. This is the most traversal-heavy phase of MCRT and the place where the DDA speedup pays back many times over.
- ▶ **Beyond Cartesian.** The same template — parametric distances to the next boundaries, take the minimum, advance one axis — generalizes to nonuniform axis-aligned grids and to spherical or cylindrical grids (one quadratic solve per shell crossing).

Take-home messages

Part I — Numba. Put `@njit` on the hot loop and every helper it calls; leave the orchestrating wrapper in plain Python unless it is itself the hot loop or wraps a prange. Use `cache=True` for persistent speedup across sessions; warm-up is only cosmetic.

Part II — Ray traversal. The baseline algorithm (Seon 2009) is correct but wastes three divisions per voxel crossing. The Amanatides–Woo DDA hoists the divisions into a one-time initialization, leaving the inner loop with only comparisons and an addition. The per-step speedup is $\sim 10\text{--}15\times$ in the division-bound regime, $\sim 3\text{--}5\times$ when division is well-pipelined — a fixed multiplier on every voxel crossing.

Together. A Numba-compiled DDA traversal is the natural inner kernel for any production-grade MCRT code. The Python/JIT boundary is crossed once per packet (or once per ray for peeling-off), and the per-voxel arithmetic is reduced to a handful of cycles.

References

- ▶ Numba documentation, <https://numba.readthedocs.io/>.
- ▶ Seon, K.-I. 2009, “Monte-Carlo simulation of the dust scattering,” *PKAS* **24**, 43–51.
- ▶ Amanatides, J. and Woo, A. 1987, “A fast voxel traversal algorithm for ray tracing,” *Proc. Eurographics '87*, 87, 3–10.
- ▶ Witt, A. N. 1977, “Multiple scattering in reflection nebulae. I. A Monte Carlo approach,” *ApJS* **35**, 1.
- ▶ Yusef-Zadeh, F., Morris, M., and White, R. L. 1984, “Bipolar reflection nebulae: Monte Carlo simulations,” *ApJ* **278**, 186.